

Inheritance and Polymorphism

Polymorphism

A class defines a type. A type defined by a subclass is called a *subtype*, and a type defined by its superclass is called a *supertype*.

Therefore, you can say that **Circle** is a subtype of **GeometricObject** and **GeometricObject** is a supertype for **Circle**.

Polymorphism means that a variable of a supertype can refer to a subtype object.

PolymorphismDemo

Polymorphism, Dynamic Binding and Generic Programming

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }
```

An object of a subtype can be used wherever its supertype value is required. This feature is known as **polymorphism**.

```
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}
```

```
class GraduateStudent extends Student{}
```

```
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}
```

Method m takes a parameter of the Object type. You can invoke it with any object.

```
class Person extends Object {  
    public String toString() {  
        return "Person";  
    }  
}
```

Polymorphism, Dynamic Binding and Generic Programming

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }
```

Polymorphism allows methods to be used generically for a wide range of object arguments. This is known as **generic programming**.

```
        public static void m(Object x) {  
            System.out.println(x.toString());  
        }  
    }
```

```
class GraduateStudent extends Student{}
```

```
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}
```

```
class Person extends Object {  
    public String toString() {  
        return "Person";  
    }  
}
```

If a method's parameter type is a superclass (e.g., Object), you may pass an object to this method of any of the parameter's subclasses (e.g., Student or String).

Polymorphism, Dynamic Binding and Generic Programming

```
public class PolymorphismDemo {  
    public static void main(String[] args) {  
        m(new GraduateStudent());  
        m(new Student());  
        m(new Person());  
        m(new Object());  
    }  
  
    public static void m(Object x) {  
        System.out.println(x.toString());  
    }  
}
```

When the method `m(Object x)` is executed, the argument `x`'s `toString` method is invoked. `x` may be an instance of `GraduateStudent`, `Student`, `Person`, or `Object`.

```
class GraduateStudent extends Student{}
```

```
class Student extends Person {  
    public String toString() {  
        return "Student";  
    }  
}
```

```
class Person extends Object {  
    public String toString() {  
        return "Person";  
    }  
}
```

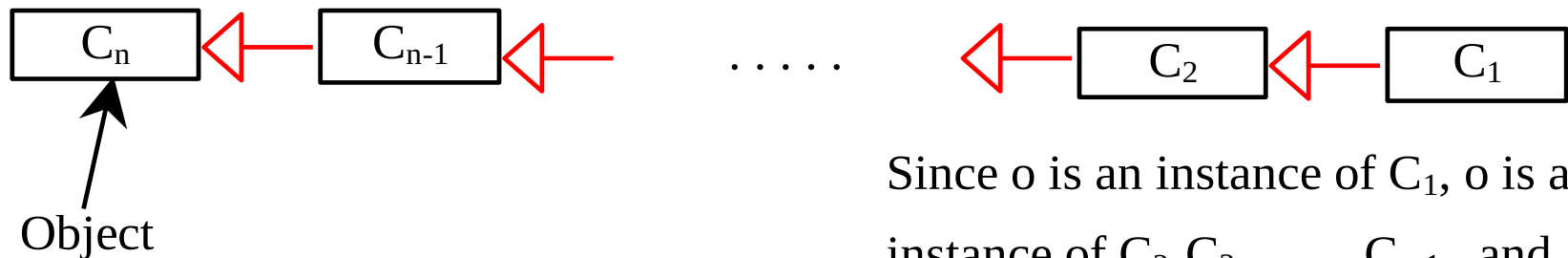
Each class has its own implementation to the `toString` method.

Which implementation is used will be determined dynamically by the Java Virtual Machine at runtime.

This capability is known as **dynamic binding**.

Dynamic Binding

Dynamic binding works as follows: Suppose an object o is an instance of classes $C_1, C_2, \dots, C_{n-1}$, and C_n , where C_1 is a subclass of C_2 , C_2 is a subclass of C_3 , ..., and C_{n-1} is a subclass of C_n . That is, C_n is the most general class, and C_1 is the most specific class. In Java, C_n is the Object class. If o invokes a method p , the JVM searches the implementation for the method p in $C_1, C_2, \dots, C_{n-1}$ and C_n , in this order, until it is found. Once an implementation is found, the search stops and the first-found implementation is invoked.



Since o is an instance of C_1 , o is also an instance of $C_2, C_3, \dots, C_{n-1}$, and C_n

Method Matching vs. Binding

Matching a method signature and binding a method implementation are two issues. The compiler finds a matching method according to parameter type, number of parameters, and order of the parameters at compilation time. A method may be implemented in several subclasses. The Java Virtual Machine dynamically binds the implementation of the method at runtime.

Casting Objects

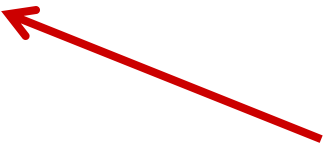
Casting can be used to convert an object of one class type to another **within an inheritance hierarchy**.

In the preceding section, the statement:

```
m(new Student()); // Implicit casting
```

assigns the object new Student() to a parameter of the Object type.
This statement is equivalent to:

```
Object o = new Student(); // Implicit casting  
m(o);
```



The statement Object o = new Student(), known as implicit casting, is legal because an instance of Student is automatically an instance of Object.

Why Casting Is Necessary?

Suppose you want to assign the object reference `o` to a variable of the `Student` type using the following statement:

```
Student b = o; //A compile (syntax) error
```

Why does the statement **`Object o = new Student()`** work and the statement **`Student b = o`** doesn't? This is because a `Student` object is always an instance of `Object`, but an `Object` is not necessarily an instance of `Student`. Even though you can see that `o` is really a `Student` object, the compiler is not so clever to know it.

To tell the compiler that `o` is a `Student` object, use an explicit casting. Enclose the target object type in parentheses and place it before the object to be cast, as follows:

```
Student b = (Student) o; // Explicit casting
```

Casting from Superclass to Subclass

Explicit casting must be used when casting an object from a superclass to a subclass. This type of casting may not always succeed.

```
Apple x = (Apple) fruit;
```

```
Orange x = (Orange) fruit;
```

The instanceof Operator

Use the `instanceof` operator to test whether an object is an instance of a class:

```
Object myObject = new Circle();  
... // Some lines of code  
/** Perform casting if myObject is an instance of  
    Circle */  
if (myObject instanceof Circle) {  
    System.out.println("The circle diameter is " +  
        ((Circle)myObject).getDiameter());  
    ...  
}
```

TIP

To help understand casting, you may also consider the analogy of fruit, apple, and orange with the Fruit class as the superclass for Apple and Orange. An apple is a fruit, so you can always safely assign an instance of Apple to a variable for Fruit. However, a fruit is not necessarily an apple, so you have to use explicit casting to assign an instance of Fruit to a variable of Apple.

The protected Modifier

- ❑ The `protected` modifier can be applied on data and methods in a class. A protected data or a protected method in a public class can be accessed by any class in the same package or its subclasses, even if the subclasses are in a different package.
- ❑ `private`, `default`, `protected`, `public`

Visibility increases
—————→
`private`, `none` (if no modifier is used), `protected`, `public`

Visibility Modifiers and Accessor/Mutator Methods

By default, the class, variable, or method can be accessed by any class in the same package.

- ❑ **public**

The class, data, or method is visible to any class in any package using qualified name (a reference and a dot notation).

- ❑ **private**

The data or methods can be accessed only by the declaring class. The get and set methods are used to read and modify private properties.

- ❑ **Protected**

The data or methods are accessible inside class where they are declared, and inside subclasses. All class in the same package can access protected member using qualified name.

- ❑ **No modifier: default members** can be accessed by all classes in the same package using qualified name

Accessibility Summary

Modifier on members in a class	Accessed from the same class	Accessed from the same package	Accessed from a subclass	Accessed from a different package
public	✓	✓	✓	✓
protected	✓	✓	✓	–
default	✓	✓	–	–
private	✓	–	–	–

The private modifier restricts access to within a class, the default modifier restricts access to within a package, and the public modifier enables unrestricted access.

```
package p1;
```

```
class C1 {  
    ...  
}
```

```
package p1;
```

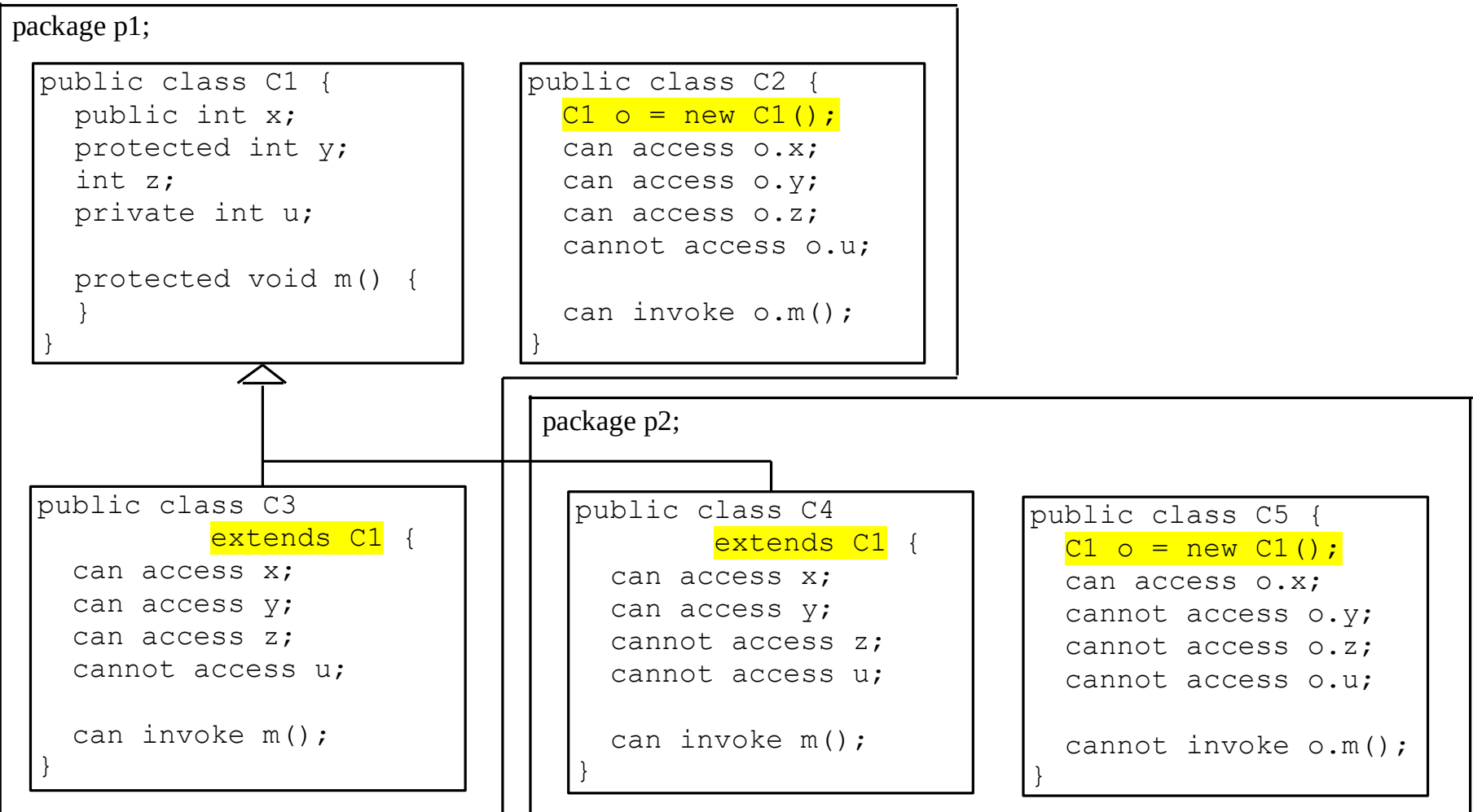
```
public class C2 {  
    can access C1  
}
```

```
package p2;
```

```
public class C3 {  
    cannot access C1;  
    can access C2;  
}
```

The default modifier on a class restricts access to within a package, and the public modifier enables unrestricted access.

Visibility Modifiers



A Subclass Cannot Weaken the Accessibility

A subclass may override a protected method in its superclass and change its visibility to public. However, a subclass cannot weaken the accessibility of a method defined in the superclass. For example, if a method is defined as public in the superclass, it must be defined as public in the subclass.

The `final` Modifier

- ❑ The `final` class cannot be extended:

```
final class Math {  
    ...  
}
```

- ❑ The `final` variable is a constant:

```
final static double PI = 3.14159;
```

- ❑ The `final` method cannot be overridden by its subclasses.

Example: Demonstrating Polymorphism and Casting

This example creates two geometric objects: a circle, and a rectangle, invokes the `displayGeometricObject` method to display the objects. The `displayGeometricObject` displays the area and diameter if the object is a circle, and displays area if the object is a rectangle.

CastingDemo

The ArrayList Class

You can create an array to store objects. But the array's size is fixed once the array is created. Java provides the ArrayList class that can be used to store an unlimited number of objects.

java.util.ArrayList<E>

```
+ArrayList()  
+add(o: E) : void  
+add(index: int, o: E) : void  
+clear(): void  
+contains(o: Object): boolean  
+get(index: int) : E  
+indexOf(o: Object) : int  
+isEmpty(): boolean  
+lastIndexOf(o: Object) : int  
+remove(o: Object): boolean  
+size(): int  
+remove(index: int) : boolean  
+set(index: int, o: E) : E
```

Creates an empty list.

Appends a new element *o* at the end of this list.

Adds a new element *o* at the specified index in this list.

Removes all the elements from this list.

Returns true if this list contains the element *o*.

Returns the element from this list at the specified index.

Returns the index of the first matching element in this list.

Returns true if this list contains no elements.

Returns the index of the last matching element in this list.

Removes the element *o* from this list.

Returns the number of elements in this list.

Removes the element at the specified index.

Sets the element at the specified index.

Generic Type

ArrayList is known as a generic class with a generic type E. You can specify a concrete type to replace E when creating an ArrayList. For example, the following statement creates an ArrayList and assigns its reference to variable cities. This ArrayList object can be used to store strings.

```
ArrayList<String> cities = new ArrayList<String>();
```

```
ArrayList<String> cities = new ArrayList<>();
```

TestArrayList

Differences and Similarities between Arrays and ArrayList

Operation	Array	ArrayList
Creating an array/ArrayList	<code>String[] a = new String[10]</code>	<code>ArrayList<String> list = new ArrayList<>();</code>
Accessing an element	<code>a[index]</code>	<code>list.get(index);</code>
Updating an element	<code>a[index] = "London";</code>	<code>list.set(index, "London");</code>
Returning size	<code>a.length</code>	<code>list.size();</code>
Adding a new element		<code>list.add("London");</code>
Inserting a new element		<code>list.add(index, "London");</code>
Removing an element		<code>list.remove(index);</code>
Removing an element		<code>list.remove(Object);</code>
Removing all elements		<code>list.clear();</code>

DistinctNumbers

Array Lists from/to Arrays

Creating an ArrayList from an array of objects:

```
String[] array = {"red", "green", "blue"};
```

```
    ArrayList<String> list = new  
ArrayList<>(Arrays.asList(array));
```

Creating an array of objects from an ArrayList:

```
String[] array1 = new String[list.size()];
```

```
list.toArray(array1);
```


max and min in an Array List

```
String[] array = {"red", "green", "blue"};
```

```
System.out.println(java.util.Collections.max(  
    new ArrayList<String>(Arrays.asList(array))));
```

```
String[] array = {"red", "green", "blue"};
```

```
System.out.println(java.util.Collections.min(  
    new ArrayList<String>(Arrays.asList(array))));
```

Shuffling an Array List

```
Integer[] array = {3, 5, 95, 4, 15, 34, 3, 6, 5};  
ArrayList<Integer> list = new  
    ArrayList<>(Arrays.asList(array));  
java.util.Collections.shuffle(list);  
System.out.println(list);
```

Designing a Class

(Coherence) A class should describe a single entity, and all the class operations should logically fit together to support a coherent purpose.

- You can use a class for students, for example, but you should not combine students and staff in the same class, because students and staff have different entities.

Designing a Class, cont.

(Separating responsibilities) A single entity with too many responsibilities can be broken into several classes to separate responsibilities.

- The classes `String`, `StringBuilder`, and `StringBuffer` all deal with strings, for example, but have different responsibilities. The `String` class deals with immutable strings, the `StringBuilder` class is for creating mutable strings, and the `StringBuffer` class is similar to `StringBuilder` except that `StringBuffer` contains synchronized methods for updating strings.

Designing a Class, cont.

Classes are designed for reuse. Users can incorporate classes in many different combinations, orders, and environments. Therefore, you should design a class that imposes no restrictions on what or when the user can do with it, design the properties to ensure that the user can set properties in any order, with any combination of values, and design methods to function independently of their order of occurrence.

Designing a Class, cont.

Provide a public no-arg constructor and override the equals method and the toString method defined in the Object class whenever possible.

Designing a Class, cont.

Follow standard Java programming style and naming conventions. Choose informative names for classes, data fields, and methods. Always place the data declaration before the constructor, and place constructors before methods. Always provide a constructor and initialize variables to avoid programming errors.

Using Visibility Modifiers

Each class can present two contracts – one for the users of the class and one for the extenders of the class. Make the fields private and accessor methods public if they are intended for the users of the class. Make the fields or method protected if they are intended for extenders of the class. The contract for the extenders encompasses the contract for the users. The extended class may increase the visibility of an instance method from protected to public, or change its implementation, but you should never change the implementation in a way that violates that contract.

Using Visibility Modifiers, cont.

A class should use the private modifier to hide its data from direct access by clients. You can use get methods and set methods to provide users with access to the private data, but only to private data you want the user to see or to modify. A class should also hide methods not intended for client use. The gcd method in the Rational class is private, for example, because it is only for internal use within the class.

Using the static Modifier

A property that is shared by all the instances of the class should be declared as a static property.

Thank you
Thank you

